DiffWAM: Learning World Action Model from Consecutive Visual Difference

Xinheng Zhang[1,2], **Haoyu Xiong**[3,4], **Chao Jing**[3,5], **Hongbang Fan**[3,4], **and Xingyu Chen**[3]

[1]College of Information Science and Electronic Engineering, Zhejiang University
[2]Polytechnic Institute, Zhejiang University
[3]Zhongguancun Academy
[4]Institute of Automation, Chinese Academy of Sciences
[5]School of Intelligent Robotics and Advanced Manufacturing Innovation, Fudan University

https://zgca-hmi-lab.github.io/DiffWAM/
https://github.com/ZGCA-HMI-Lab/DiffWAM

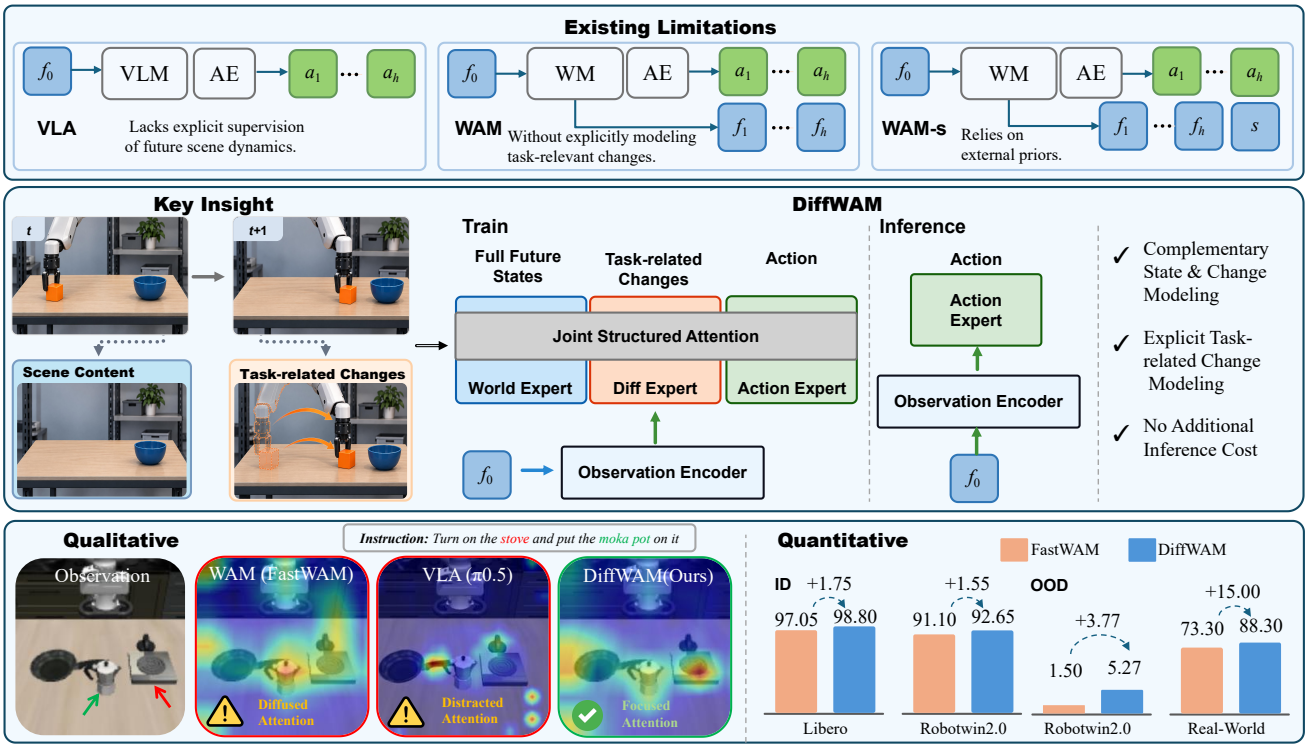


Figure 1. **Overview of DiffWAM. Key insight and method:** Consecutive observations contain persistent scene content and task-related changes, motivating complementary state and difference modeling. During training, the World, Diff, and Action Experts predict full future states, consecutive visual differences, and actions under structured attention. Only the Observation Encoder and Action Expert are retained at inference, introducing no additional inference cost. **Existing limitations:** The illustrated VLA paradigm lacks explicit future-state supervision; WAM predicts future scenes without explicitly distinguishing task-related changes; and WAM-s relies on potentially unreliable external priors. **Qualitative and quantitative evidence:** Qualitative attention visualizations suggest that DiffWAM focuses more on task-relevant regions, while quantitative comparisons show higher success rates than FastWAM on in-distribution LIBERO and RoboTwin 2.0 benchmarks and out-of-distribution RoboTwin 2.0 and real-world evaluations.

Abstract

World Action Models (WAMs) have emerged as an important paradigm in robot learning, jointly predicting future scene states and robot actions from current observations and instructions. However, future states entangle task-related changes with large amounts of task-irrelevant content, such as static backgrounds, which weakens the model's ability to capture changes relevant to task execution. To address this, we propose DiffWAM, a Mixture-of-Experts (MoE) framework that enhances world representations by jointly learning future-state prediction and consecutive visual difference prediction. We define consecutive visual differences as differences between the latent representations of consecutive observations, providing an explicit learning signal for temporal variation. The framework comprises a World Expert for future frame prediction, an Action Expert for action generation, and a Diff Expert for predicting these latent differences along the trajectory. A structured unidirectional attention mask conditions the Diff, World, and Action Experts solely on the current-frame representation while blocking attention between experts. This design allows both the World and Diff Experts to be removed during inference, introducing no additional inference overhead. DiffWAM achieves state-of-the-art performance on the LIBERO and RoboTwin 2.0 benchmarks and substantially improves over FastWAM on real-world manipulation tasks. Specifically, it attains average success rates of 98.8%, 92.65%, and 88.3%, outperforming the state-of-the-art baseline (i.e., FastWAM) by 1.75, 1.55, and 15.0 percentage points, respectively.

1 Introduction

Learning task-related representations is crucial for robotic manipulation, as a robot must understand scene content and changes relevant to task execution. Two main paradigms have emerged to this end. Vision Language Action models (VLAs) [1, 2] map observations and instructions directly to actions, which is simple and efficient but lacks explicit modeling of scene evolution during task execution [3, 4, 5, 6, 7, 8, 9]. In contrast, World Action Models (WAMs) [10, 11], building on the predictive world modeling paradigm [12], learn by predicting future scene states alongside robot actions, thereby acquiring a predictive understanding of task-related scene dynamics [13, 14, 15, 16].

Despite recent progress in WAMs [17, 18, 19, 20, 21], a key question remains: *is future-state prediction alone sufficient to learn world representations that capture task-related changes?* As illustrated in Figure 1, much of the scene content persists across consecutive observations, while task execution involves changes such as robot motion and object interactions. The full-state prediction objective supervises these changes together with persistent content, without providing a separate learning signal for temporal variation. Thus, the representation of task-related changes relies on what the model learns implicitly from predicting complete future states.

This observation suggests a principle of visual foresight: a WAM should model not only what the future scene will look like, but also how it changes over time. Future-state prediction captures scene structure, object appearance, and spatial relationships, while consecutive visual difference prediction provides a separate target for temporal variation. We define a *consecutive visual difference* as the difference between the latent representations of two consecutive observations. Predicting this quantity, also referred to as *diff prediction*, provides explicit supervision for temporal variation without requiring additional motion annotations. By jointly modeling future states and their consecutive visual differences, a WAM can develop a more comprehensive world representation that integrates scene understanding with task-related dynamics for action generation.

In this paper, we propose DiffWAM, a Mixture-of-Experts (MoE) framework that jointly learns future-state prediction, visual difference prediction, and action generation. A shared Observation Encoder encodes the current visual observation into world representations, which are used by a World Expert to predict future states, a Diff Expert to predict differences between consecutive frame latents, and an Action Expert to generate actions. A structured unidirectional attention mask allows each expert to attend to the current-observation representations while blocking attention between experts. This architecture integrates state and visual difference modeling into the world representations used for action generation, while keeping the prediction pathways separate. At inference, both the World and Diff Experts are removed, retaining only the Observation Encoder and Action Expert without additional inference overhead. Across LIBERO [22], RoboTwin 2.0 [23], and real-world manipulation tasks, DiffWAM achieves average success rates of 98.8%, 92.65%, and 88.3%, respectively, outperforming FastWAM [15] by 1.75, 1.55, and 15.0 percentage points.

Our contributions are summarized as follows:

- We introduce joint future-state and visual difference modeling in WAMs, enabling world representations to capture both scene content and task-related dynamics for action generation.
- We propose DiffWAM, a Mixture-of-Experts framework with a shared Observation Encoder and dedicated World, Diff, and Action Experts. A structured unidirectional attention mask separates the expert pathways, allowing the World and Diff Experts to be removed at inference without additional overhead.
- We validate DiffWAM on LIBERO, RoboTwin 2.0, and real-world manipulation tasks, demonstrating its effectiveness across simulated and physical settings. DiffWAM reduces the optimization steps required to reach 95% average success on LIBERO by 28.1%. Ablation studies validate the benefits of joint state and visual difference modeling.

2 Related Work

Existing embodied policy models can be broadly grouped into three paradigms according to their model outputs and supervision: VLA, WAM with Future Prediction, and WAM with Structured Signal Prediction. We detail these three paradigms below.

2.1 VLA

Vision-Language-Action models directly generate actions from visual observations and language instructions, learning representations for action generation without an explicit future-state prediction objective. This paradigm formulates the problem as learning a conditional action policy:

$$\pi_\theta(a_{1:h} \mid o_0, l)\,, \tag{1}$$

where o_0 denotes the current observation, l the language instruction, and $a_{1:h}$ the action sequence over a horizon of h steps. Such policies typically couple a VLM backbone with an action expert, directly decoding actions from observation and instruction [24, 25].

Representative works following this paradigm include RT-2 [4], OpenVLA [5], $\pi 0$ [7], Octo [26], Diffusion Policy [27], RDT-1B [28], and Xiaomi-Robotics-0 [29]. These methods directly map observations to actions, and VLM-based approaches can benefit from pretrained semantic representations. However, learning directly from demonstrations may also capture spurious correlations, potentially contributing to compounding errors and reduced generalization [30].

2.2 WAM with Future Prediction

WAMs with future prediction jointly model future visual states and actions, extending action generation with an explicit representation of scene evolution. Conditioned on the current observation and language instruction, they model the joint distribution:

$$p_\theta(o_{1:h}, a_{1:h} \mid o_0, l)\,, \tag{2}$$

where $o_{1:h}$ denotes the predicted future frames and $a_{1:h}$ the corresponding action sequence. Rather than modeling environmental dynamics implicitly, these methods explicitly predict future frames to ground action generation [31, 32].

Representative works following this paradigm include Motus [13], LingBot-VA [14], Fast-WAM [15], GigaWorld-Policy [33], IRASim [34], VideoVLA [35], Video Prediction Policy [36], and WorldVLA [37]. Future visual prediction connects scene evolution with action generation [38, 39]. Nevertheless, accurately predicting temporally coherent future frames in high-dimensional pixel or latent space remains challenging. For methods that condition action decoding directly on predicted future states, prediction errors may propagate into action generation [40]. Moreover, predicting complete future states does not explicitly distinguish persistent scene content from inter-frame visual differences, leaving temporal variation to be learned implicitly within the state prediction objective. This can make it difficult to emphasize task-related changes over visually salient but task-irrelevant details [41].

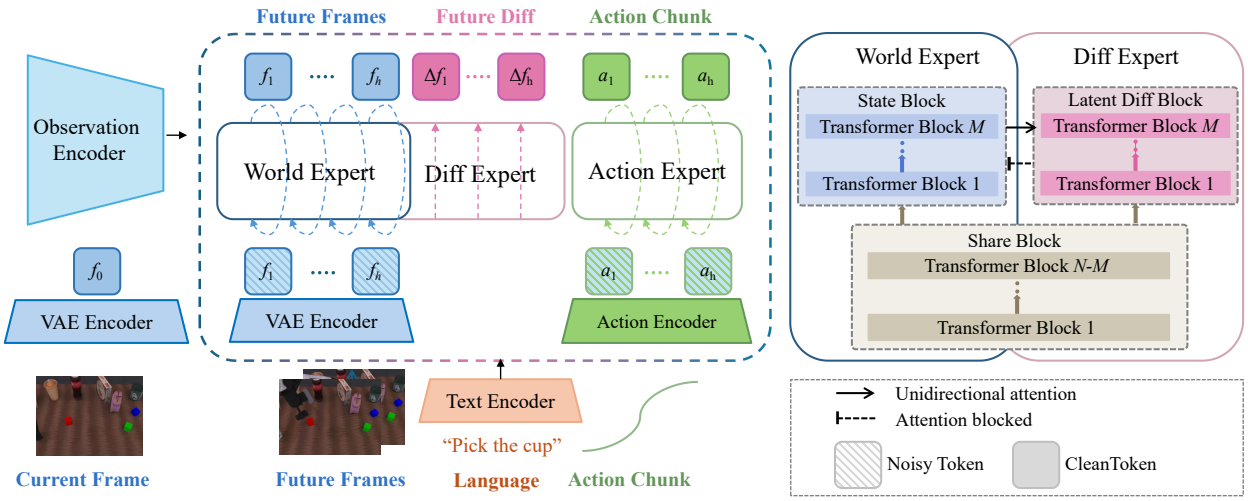


Figure 2. **Framework of DiffWAM.** The Observation Encoder provides current-observation representations to the World, Diff, and Action Experts. The World and Diff Experts receive the same noised future-frame latents, share their first $N-M$ layers, and use independent final M layers for state generation through denoising and latent-difference prediction through regression, respectively. Structured unidirectional attention separates the expert pathways. At inference, both visual prediction experts are removed, retaining only the Observation Encoder and Action Expert without additional inference overhead.

2.3 WAM with Structured Signal Prediction

Beyond future-state prediction, some WAMs incorporate structured prediction targets, such as depth, semantic segmentation, optical flow, or pretrained visual features, to model additional aspects of the scene. Formally, such methods introduce a structured representation s and model

$$p_\theta(o_{1:h}, s, a_{1:h} \mid o_0, l)\,, \tag{3}$$

where $o_{1:h}$ denotes the predicted future frames, $a_{1:h}$ the corresponding actions, and s an auxiliary structured signal, such as a depth map, semantic segmentation, optical flow [16, 42], or a pretrained feature field (e.g., DINOv3 features [43]). These targets capture geometric, semantic, or motion-related information, depending on the chosen signal. By supplementing future-state prediction with such structure, these methods introduce additional inductive biases into visual modeling.

Representative works include X-WAM [44], AIM [45], AGRA [46], DeFI [47], and MoLA [48]. These approaches specify particular geometric, semantic, or motion-related quantities as prediction targets. Their modeling emphasis therefore depends on the chosen signal, and targets obtained from external estimators may contain estimation errors. In contrast, DiffWAM directly models differences between consecutive frame latents alongside future states, integrating state and visual difference prediction within a WAM.

These paradigms provide different ways to connect visual modeling with action generation. Our work extends future-predictive WAMs by explicitly modeling both future states and inter-frame latent differences, aiming to capture scene content and task-related dynamics within a shared world representation.

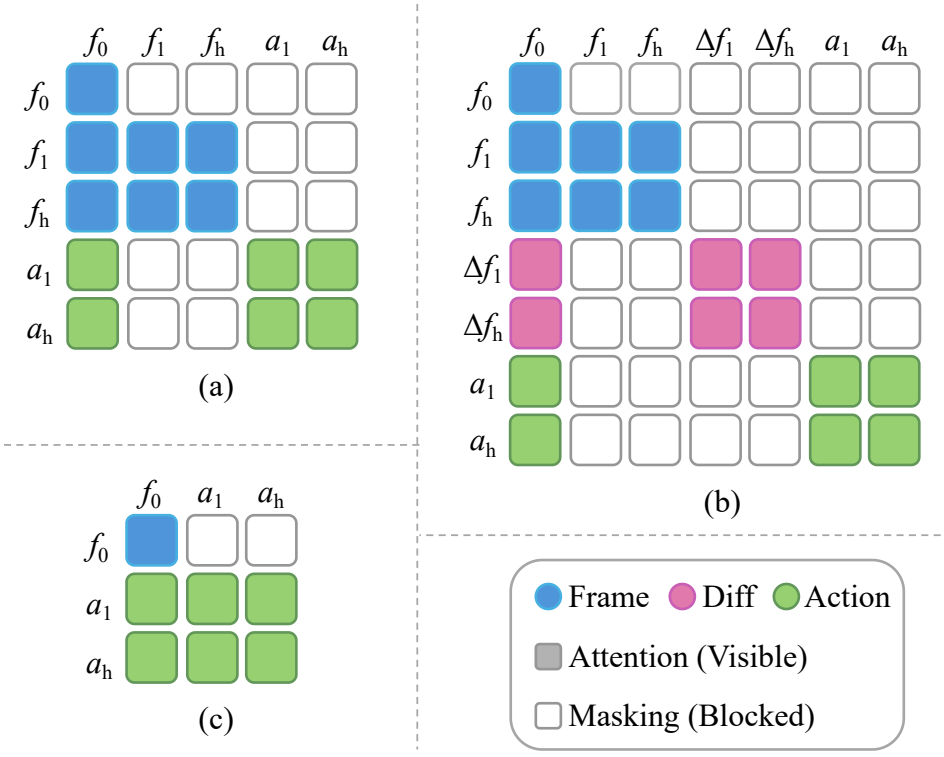


Figure 3. Structured attention masks in DiffWAM. (a) Attention in the first $N - M$ layers, where the World and Diff Experts share visual processing layers. (b) Attention in the last M layers, with independent state and diff prediction pathways. (c) Observation and action pathways retained at inference. Each pathway attends to observation representations and its own tokens, while cross-attention between expert pathways is blocked.

3 Method

3.1 Problem Formulation

As illustrated in Figure 2, DiffWAM jointly models future states, inter-frame visual differences, and actions conditioned on the current observation o_0 and a language instruction l. Let f_i denote the VAE latent representation of observation o_i, T the number of future visual states, and H the action chunk length. The joint modeling objective is expressed in latent space as:

$$p_\theta(f_{1:T}, \Delta f_{1:T}, a_{1:H} \mid f_0, l)\,, \tag{4}$$

where $f_{1:T}$ denotes the future visual latents, $a_{1:H}$ the action chunk, and $\Delta f_{1:T}$ the consecutive visual differences represented in latent space (labeled *Future Diff* in Figure 2), defined as

$$\Delta f_i = f_i - f_{i-1}, \qquad i = 1, \ldots, T. \tag{5}$$

We use *visual difference prediction* for this auxiliary task and *diff prediction* as its shorthand; the Diff Expert predicts the latent differences Δf_i. Future-state prediction captures scene content, while visual difference prediction explicitly supervises inter-frame variation. Jointly learning these complementary prediction tasks shapes the current-observation representation used for action generation. The joint objective does not require the Action Expert to consume predicted future states or latent

differences; each expert is conditioned on the current-observation representation and language instruction. Latent differences describe visual variation and may also include nuisance changes, rather than exclusively encoding changes caused by robot actions.

3.2 Model Architecture

Overview. DiffWAM jointly learns future-state prediction, visual difference prediction, and action generation to enhance world representations. During training, the World and Diff Experts model future states and latent differences, respectively, while the Action Expert learns to generate actions from the current-observation representation. During inference, both the World and Diff Experts are removed, retaining only the Observation Encoder and Action Expert without additional inference overhead.

Architecture. As illustrated in Figure 2, DiffWAM is a Mixture-of-Experts framework with a shared Observation Encoder and dedicated World, Diff, and Action Experts. The Observation Encoder and each expert contain N Transformer layers. The World and Diff Experts share parameters in their first $N - M$ layers and use independent parameters in their last M layers. The Observation Encoder maps the current visual latents into world representations that condition all three experts. The World Expert predicts future states, the Diff Expert predicts inter-frame latent differences, and the Action Expert generates the action chunk. A structured attention mask separates the expert pathways while allowing each expert to access the current-observation representation. Parameter sharing between the World and Diff Experts does not introduce cross-attention between their independent prediction pathways.

Input Encoding. DiffWAM inherits the pretrained text encoder and video VAE from Wan2.2-5B [49]. The task instruction l is encoded with the T5 text encoder [50] and provides language conditioning through cross-attention. Images from multiple camera views are concatenated before being mapped into visual latents by the pretrained VAE [51]. The clean current-frame latents f_0 are processed by the Observation Encoder to provide the shared conditioning representation. During training, both the World and Diff Experts receive the same noised future-frame latents as input. The World Expert learns future-state generation through denoising, while the Diff Expert directly regresses latent differences, with targets computed from consecutive ground-truth visual latents. Actions $a_{1:H}$ are separately noised and embedded for the Action Expert.

World Expert. The World Expert models future states conditioned on the current-observation representation supplied by the Observation Encoder and the language instruction. Noised future-frame latents are processed by the first $N - M$ layers shared with the Diff Expert, followed by M independent state-prediction layers. The World Expert learns the velocity field used for future-state generation through flow matching. Its prediction pathway is used during training and is removed at inference; the Observation Encoder is retained to provide the representation for action generation.

Diff Expert. The Diff Expert processes the same noised future-frame latents through the shared first $N - M$ layers, followed by an independent M-layer Latent Diff Block (LDB). Conditioned on the current-observation representation and language instruction, it directly regresses inter-frame latent differences rather than generating future states through denoising. Its independent pathway does not attend to the World or Action Expert's tokens, and its outputs are not passed to these experts. The regression targets are differences between consecutive ground-truth frame latents, including the transition from the current observation to the first future state. Visual difference modeling influences the shared representation during training, while the Diff Expert is removed at inference.

Action Expert. The Action Expert generates the action chunk through flow matching, conditioned on the current-observation representation supplied by the Observation Encoder and the language instruction. It does not directly receive predicted future states or diff outputs. State and visual difference modeling improve the representation used by the action pathway through joint training. At inference, the Action Expert operates with the Observation Encoder after both visual prediction experts have been removed.

Structured Attention Mask. As illustrated in Figure 3, a structured unidirectional attention mask controls information flow between the current-observation pathway and the experts. Current-observation tokens do not attend to future-state, diff, or action tokens. Each expert may attend to the current-observation representation and tokens within its own pathway, while attention between different expert pathways is blocked. These restrictions concern interactions between visual and action token groups; language conditioning is supplied separately through cross-attention.

(a) In the first $N - M$ layers, the visual and action pathways follow a first-frame-conditioned attention scheme [15]: current-

frame tokens attend to themselves, noised future-frame tokens attend to the current-frame representation and their own tokens, and action tokens attend to the current-frame representation and their own tokens. Future-frame and action tokens do not attend to each other. The World and Diff Experts share the visual processing layers at this depth.
(b) In the last M layers, the World and Diff Experts use independent prediction pathways. Diff tokens attend to the current-frame representation and their own tokens, while the state and action pathways retain the attention restrictions in (a). Blocking cross-attention between these pathways prevents the Diff Expert from reading the World or Action Expert's token representations and prevents its outputs from entering their prediction pathways. This restriction does not exclude the noised future-frame input shared by the World and Diff Experts.
(c) At inference, only the current-observation and action pathways are retained. The Action Expert generates $a_{1:H}$ from the current-observation representation and language instruction without invoking future-state or diff prediction, introducing no additional overhead from the visual prediction experts.

3.3 Joint Training Objective

DiffWAM is trained with three losses: video flow matching [52] for future-state prediction, squared-error latent regression for difference prediction, and action flow matching [53] for action generation. Both flow branches are conditioned on the current-observation representation derived from f_0 and the language instruction l [54]. In the following equations, conditioning on f_0 denotes access to this representation through the Observation Encoder.

World Flow Matching Loss. For each visual prediction step $i \in \{1, \dots, T\}$, let f_i denote the target future-frame latent and $\epsilon_i \sim \mathcal{N}(0, \mathbf{I})$ the sampled noise prior. We construct the interpolated sample along the probability path as $f_{i,\tau} = (1-\tau)\epsilon_i + \tau f_i$ with $\tau \in [0, 1]$. Conditioned on the current-observation representation and the language instruction l, the World Expert is supervised to predict the corresponding velocity field:

$$\mathcal{L}_{\text{world}} = \mathrm{E}_{i,\tau,\epsilon_i,f_i}\left[\left\|v_\theta(f_{i,\tau}, \tau \mid f_0, l) - (f_i - \epsilon_i)\right\|_2^2\right], \tag{6}$$

where v_θ is the predicted velocity field parameterized by θ.

Diff Regression Loss. At each visual prediction step $i \in \{1, \dots, T\}$, the Diff Expert receives the same noised future-frame latent $f_{i,\tau}$ as the World Expert and directly produces a prediction $\widehat{\Delta f}_i$ of the latent difference between consecutive observations. We use squared-error supervision in latent space, as also used for latent prediction in DINO-WM [55]. Here, the regression target $\Delta f_i = f_i - f_{i-1}$ is obtained by differencing consecutive ground-truth frame latents:

$$\mathcal{L}_{\text{diff}} = \mathrm{E}_{i,\tau,\epsilon_i,f_{i-1},f_i}\left[\left\|\widehat{\Delta f}_i - (f_i - f_{i-1})\right\|_2^2\right], \tag{7}$$

where $\widehat{\Delta f}_i$ is the LDB output at visual prediction step i, and $f_i - f_{i-1}$ is the target latent difference. The case $i = 1$ uses the difference between the first future-frame latent and the current-frame latent.

Action Flow Matching Loss. Analogously, for each action step $i \in \{1, \dots, H\}$, let a_i be the ground-truth action and $\epsilon'_i \sim \mathcal{N}(0, \mathbf{I})$. We construct the interpolated sample as $a_{i,\tau} = (1-\tau)\epsilon'_i + \tau a_i$. Conditioned on the current-observation representation and the language instruction l, the Action Expert is trained to recover the target velocity field:

$$\mathcal{L}_{\text{action}} = \mathrm{E}_{i,\tau,\epsilon'_i,a_i}\left[\left\|u_\phi(a_{i,\tau}, \tau \mid f_0, l) - (a_i - \epsilon'_i)\right\|_2^2\right], \tag{8}$$

where u_ϕ is the action velocity field parameterized by ϕ. The Action Expert attends to the current-observation representations provided by the Observation Encoder, without directly receiving future-state or diff predictions. State and visual difference prediction influence action generation through joint training of the shared Observation Encoder.

Overall Objective. The complete training objective is a weighted combination of the three terms:

$$\mathcal{L} = \mathcal{L}_{\text{world}} + \lambda_{\text{diff}}\mathcal{L}_{\text{diff}} + \lambda_{\text{action}}\mathcal{L}_{\text{action}}, \tag{9}$$

where λ_{diff} and λ_{action} are hyperparameters balancing the three losses.

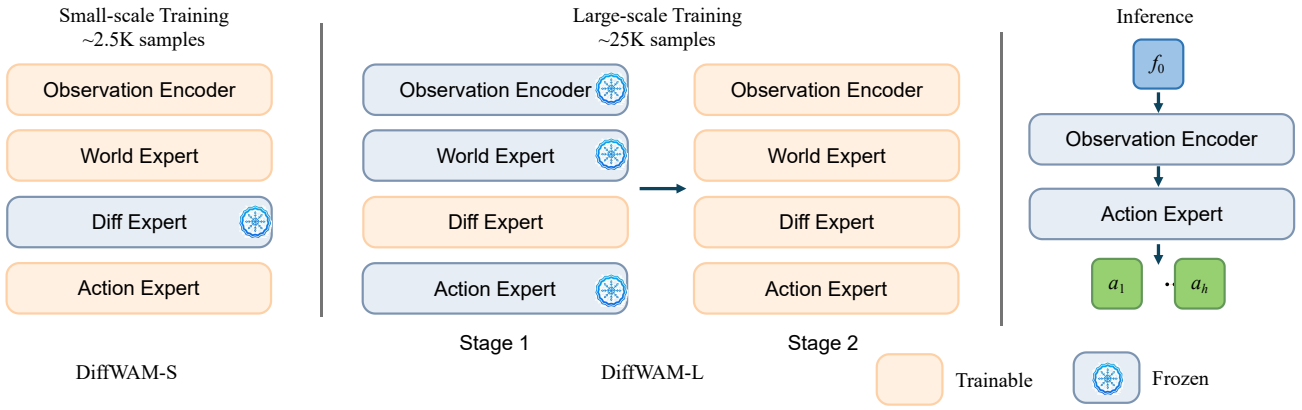


Figure 4. Training configurations and shared inference pathway of DiffWAM. DiffWAM-S freezes the independent LDB while retaining difference prediction during small-scale training. DiffWAM-L first trains the independent Diff Expert parameters with the remaining modules frozen, then jointly trains all modules. Orange denotes trainable modules, and blue with snowflake symbols denotes frozen modules; in DiffWAM-S, the shared World–Diff layers remain trainable. Both configurations retain only the Observation Encoder and Action Expert at inference.

3.4 Training Configurations and Inference

We instantiate DiffWAM in two training configurations, DiffWAM-S for small-scale training and DiffWAM-L for large-scale training. Both configurations use the architecture and joint objective described above, with different parameter update schedules. Figure 4 illustrates these configurations and their shared inference pathway.

DiffWAM-S. For small-scale training, we freeze the independent M-layer LDB of the Diff Expert while jointly optimizing the Observation Encoder, World Expert, and Action Expert, including the visual layers shared by the World and Diff Experts. Freezing the LDB helps reduce overfitting to noise in small datasets while accelerating training by reducing the number of parameters to update. The diff loss remains active: its gradients propagate through the fixed LDB into the shared visual layers and Observation Encoder. Thus, difference prediction continues to shape the representation used for action generation even though the LDB parameters are frozen.

DiffWAM-L. For large-scale training, we adopt a two-stage procedure. In the first stage, the Observation Encoder, World Expert, and Action Expert are frozen, and only the independent Diff Expert parameters are optimized using the diff regression loss. The visual layers shared with the frozen World Expert also remain frozen. In the second stage, all modules are jointly optimized with the full objective, learning future-state prediction, difference prediction, and action generation together. This schedule allows the Diff Expert to first adapt to difference prediction with a fixed visual representation, before using the larger dataset to jointly refine the shared representation and all three experts.

Inference. Both configurations use the same action-generation pathway. The current observation is encoded by the Observation Encoder, and the Action Expert generates the action chunk conditioned on this representation and the language instruction. The World and Diff Experts are removed, so future-state and visual difference prediction introduce no additional inference overhead. The two configurations therefore differ in their training schedules while retaining the same inference architecture.

4 Experiments

4.1 Implementation Details

The Observation Encoder and the World, Diff, and Action Experts each contain N Transformer layers. The World and Diff Experts use the Wan2.2-5B Transformer architecture [49], with shared parameters in the first $N - M$ layers and separate parameters in the last M layers. The Action Expert follows the same architecture but uses a reduced hidden dimension of $d_a = 1024$. The complete model contains approximately 7B parameters. Further architectural, optimization, and inference details are provided in Appendix. The additional clean-to-random study starts task training without a task-trained FastWAM checkpoint. Separately, the continued-training control initializes both methods from the same task-trained FastWAM checkpoint and trains each for one additional epoch on clean demonstrations. We distinguish these two initialization protocols throughout the analysis.

4.2 Experiment Setup

We evaluate DiffWAM on LIBERO [22], RoboTwin 2.0 [23], and three real-world manipulation tasks on the ALOHA-AgileX platform. LIBERO covers 40 single-arm tasks across four suites, while RoboTwin 2.0 evaluates 50 bimanual tasks under clean and randomized conditions. We conduct 100 evaluation rollouts per task for LIBERO and the full-data RoboTwin 2.0 evaluation. For the additional clean-to-random evaluation, we evaluate each DiffWAM policy over 30 rollouts per task in each condition. For the real-world evaluation, we use approximately 1.1k demonstration trajectories in total across three tasks (Place Pens, Store Block, and Stack Cups), train for 63k optimization steps, and report the execution success rate over 20 trials per task for each method (60 trials per method in total). Main-table results are reported for individual training runs. Appendix B.3 additionally reports results from three independent training runs with different random seeds on LIBERO and the full-data and clean-only RoboTwin 2.0 settings. Each run produces a separately trained model that is evaluated using the corresponding benchmark protocol; we report the mean and sample standard deviation across these three runs to characterize variability across independently trained models. Complete benchmark and evaluation protocols are described in the appendix.

4.3 Main Results

This section evaluates DiffWAM on LIBERO, RoboTwin 2.0, and real-world manipulation tasks by comparing it with representative VLA and WAM methods and analyzing its different training configurations and training efficiency. Additional experimental results and detailed analyses are provided in the Appendix.

Simulation Results

We evaluate DiffWAM-S on LIBERO and DiffWAM-L on RoboTwin 2.0, using the training configurations described in Section 3.4, and compare them with representative VLA and WAM methods. The results are summarized in Table 1 and Table 2.

For LIBERO, we report DiffWAM-S, the small-scale configuration described in Section 3.4. It freezes the independent LDB parameters while retaining the diff loss and its gradient propagation into the shared backbone. As shown in Table 1, DiffWAM-S achieves an average success rate of 98.8%, improving upon FastWAM by 1.75 percentage points. The improvement is most pronounced on LIBERO-Goal and LIBERO-Long, where DiffWAM-S reaches 99.0% and 98.4%, outperforming FastWAM by 1.2 and 6.0 points, respectively. DiffWAM-S achieves the highest average success rate among the methods in the table, exceeding WLA (98.6%) and LingBot-VA (98.45%), with particularly clear gains over FastWAM on goal-conditioned and long-horizon tasks.

For RoboTwin 2.0, we report DiffWAM-L, the large-scale configuration described in Section 3.4. It first trains the independent LDB with the remaining modules frozen, then jointly optimizes all modules with the full objective. As shown in Table 2, DiffWAM-L obtains an average success rate of 92.65% on RoboTwin 2.0, outperforming the reproduced FastWAM baseline by 1.55 percentage points. The gain is consistent across both evaluation conditions, increasing performance from

Method	Spatial	Object	Goal	Long	Avg.
VLA					
OpenVLA (Kim et al. 2024)	84.70	88.40	79.20	53.70	76.50
OpenVLA-O (Kim et al. 2025)	97.60	98.40	97.90	94.50	97.10
MemoryVLA (Shi et al. 2026)	98.40	98.40	96.40	93.40	96.65
π0.5 (Black et al. 2025)	98.80	98.20	98.00	92.40	96.85
GROOT-N1 (Bjork et al. 2025)	94.40	97.60	93.00	90.60	93.90
WAM					
Motus (Bi et al. 2025)	96.80	99.80	96.60	97.60	97.70
Lingbot-VA (Li et al. 2026)	98.50	99.60	97.20	**98.50**	98.45
FastWAM (Yuan et al. 2026)	98.20	99.80	97.80	92.40	97.05
ImageWAM (Zhang et al. 2026)	97.20	99.20	98.80	98.40	98.40
WLA (Yang et al. 2026)	**99.00**	**100.00**	97.80	97.60	98.60
DiffWAM-S (Ours)	98.00	99.80	**99.00**	98.40	**98.80**

Table 1. Main results on LIBERO. We use DiffWAM-S due to the small size of the LIBERO training dataset. FastWAM results are from our evaluation; its average success rate is 97.05%.

Method	Clean	Random	Avg.
VLA			
π0 [7]	65.92	58.80	62.36
X-VLA [59]	72.08	72.84	72.46
π0.5 [8]	82.74	76.76	79.75
ABot-M0 [60]	86.60	85.08	85.84
QwenVLA [61]	86.10	87.20	86.65
WAM			
Motus [13]	88.66	87.02	87.84
Lingbot-VA [14]	92.90	91.50	92.20
FastWAM [15]	91.88*	90.32*	91.10*
X-WAM [44]	89.76	90.68	90.22
WLA [58]	92.94	90.02	91.48
DiffWAM-L (Ours)	**93.28**	**92.02**	**92.65**

Table 2. Main results on RoboTwin 2.0. We use DiffWAM-L due to the large size of the RoboTwin 2.0 training dataset. '*' denotes actual reproduction results.

91.88% to 93.28% in clean scenes and from 90.32% to 92.02% under scene randomization. DiffWAM-L also surpasses the strongest reported baselines in the table: its average success rate exceeds LingBot-VA (92.20%) and WLA (91.48%), while achieving the highest results in both the clean and randomized settings. Taken together, the results across both simulation benchmarks support our hypothesis that explicitly modeling consecutive visual differences yields more task-relevant representations for complex manipulation.

Method	Embodied PT	Clean	Random	Avg.
With Embodied Pretraining				
π_0 [7]	Yes	46.4	16.3	31.35
X-VLA [59]	Yes	68.0	20.9	44.45
$\pi_{0.5}$ [8]	Yes	70.7	46.0	58.35
Without Embodied Pretraining				
StarVLA [62]	No	46.5	3.2	24.85
AHA-WAM [63]	No	64.3	3.2	33.75
FastWAM† [15]	No	**70.2**	1.50	35.85
DiffWAM (Ours)	No	69.7	**5.27**	**37.47**

Table 3. **Clean-to-random generalization on RoboTwin 2.0.** Success rates (%) for policies trained on clean demonstrations and evaluated in clean and randomized environments without further adaptation. Embodied PT denotes additional embodied pretraining. External baseline values and their pretraining groups follow the comparison reported in PINT [64]; † denotes our own FastWAM evaluation, rather than the FastWAM reproduction reported there. Our clean-only runs do not start from a task-trained FastWAM checkpoint. Avg. is the mean of Clean and Random. For DiffWAM, each condition comprises 30 trials per task; Clean, Random, and Avg. are computed from the unrounded success rates before display rounding. Bold indicates the best value within the displayed non-pretrained group.

Clean-to-Random Generalization

To evaluate transfer beyond the training distribution, we train on clean RoboTwin 2.0 demonstrations and directly evaluate under randomized conditions without further adaptation. Unlike the full-data experiment in Table 2, randomized demonstrations are excluded from these task-training runs. This study starts task training without a task-trained FastWAM checkpoint and is separate from the continued-training control below. As shown in Table 3, DiffWAM achieves 5.27% success, compared with 1.50% for our FastWAM baseline, an absolute gain of 3.77 percentage points. Clean success decreases slightly from 70.2% to 69.7%, while the mean of Clean and Random improves from 35.85% to 37.47%. It also exceeds the 3.2% reported for StarVLA and AHA-WAM among the selected methods without embodied pretraining. These results show improved transfer under the evaluated RoboTwin distribution shift, with a small trade-off in clean performance. External results provide context, while our FastWAM evaluation is the directly compared baseline.

Real-World Results

We use the DiffWAM-S configuration for real-world evaluation on three manipulation tasks: Place Pens, Store Block, and Stack Cups. The training dataset contains approximately 1.1k demonstration trajectories in total, and the model is trained for 63k optimization steps. Each method is evaluated over 20 trials per task, totaling 60 trials across the three tasks. Figure 5 shows the robot setup, task execution sequences, and success rates. DiffWAM-S achieves success rates of 90%, 85%, and 90% on the three tasks, respectively, compared with 80%, 70%, and 70% for FastWAM. Averaged across the three tasks, DiffWAM-S achieves 88.3% success versus 73.3% for FastWAM, an absolute improvement of 15.0 percentage points. These results support the benefits of explicitly modeling consecutive visual differences across real-world manipulation tasks involving different objects and motions.

Comparison of Different Configurations

We compare DiffWAM-L and DiffWAM-S across different training data scales. We consider two data regimes: small-scale training on LIBERO and RoboTwin-Clean, with approximately 2k and 2.5k demonstration trajectories, respectively, and large-scale training on RoboTwin-Full, with approximately 27k demonstration trajectories.

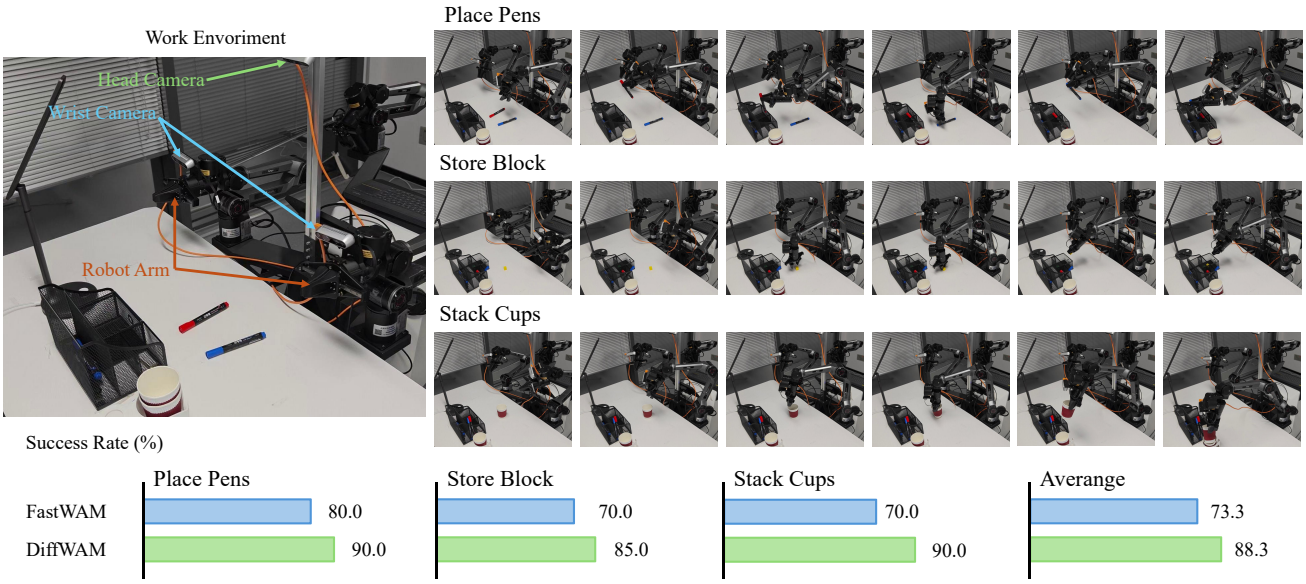


Figure 5. Real-world setup, task execution sequences, and success rates. Left: the robot workspace and camera placement. Right: example executions of Place Pens, Store Block, and Stack Cups. Bottom: success rates of FastWAM and DiffWAM-S across the three tasks. We use approximately 1.1k demonstration trajectories in total and train DiffWAM-S for 63k steps. Each method is evaluated over 20 trials per task, totaling 60 trials across the three tasks.

Method	Spatial	Object	Goal	Long	Avg.
FastWAM [15]	98.20	99.80	97.80	92.40	97.05
DiffWAM (M=5)	**99.60**	**100.00**	97.20	96.00	98.20
DiffWAM (M=10)	98.00	99.80	**99.00**	**98.40**	**98.80**
DiffWAM (M=15)	98.20	99.80	97.40	97.00	98.10
DiffWAM (M=20)	97.00	**100.00**	98.40	98.00	98.35

Table 4. Ablation study of LDB layers M on LIBERO. All DiffWAM entries use the DiffWAM-S configuration.

Architecture	Spatial	Object	Goal	Long	Avg.
World Expert	**98.20**	99.80	97.80	92.40	97.05
World Expert + Diff Loss	97.80	**100.00**	97.40	95.80	97.75
World Expert + Diff Expert	98.00	99.80	**99.00**	**98.40**	**98.80**

Table 5. Architecture ablation on LIBERO. World Expert uses the measured FastWAM results, World Expert + Diff Loss adds difference supervision without a separate Diff Expert, and World Expert + Diff Expert uses the DiffWAM-S results. The Observation Encoder and Action Expert are retained in all configurations.

As shown in Table 7, DiffWAM-L achieves 97.70% success on LIBERO, compared with 97.05% for FastWAM, while DiffWAM-S reaches 98.80%. On RoboTwin-Clean, DiffWAM-L obtains 87.72%, below the FastWAM baseline of 88.47%, whereas DiffWAM-S achieves 93.13%. DiffWAM-S therefore outperforms DiffWAM-L by 1.10 and 5.41 percentage points

Visual Experts	Spatial	Object	Goal	Long	Avg.
Only World Expert	98.20	99.80	97.80	92.40	97.05
Only Diff Expert	**99.20**	**100.00**	95.80	89.80	96.20
World + Diff Expert	98.00	99.80	**99.00**	**98.40**	**98.80**

Table 6. Ablation study of visual expert configurations on LIBERO.

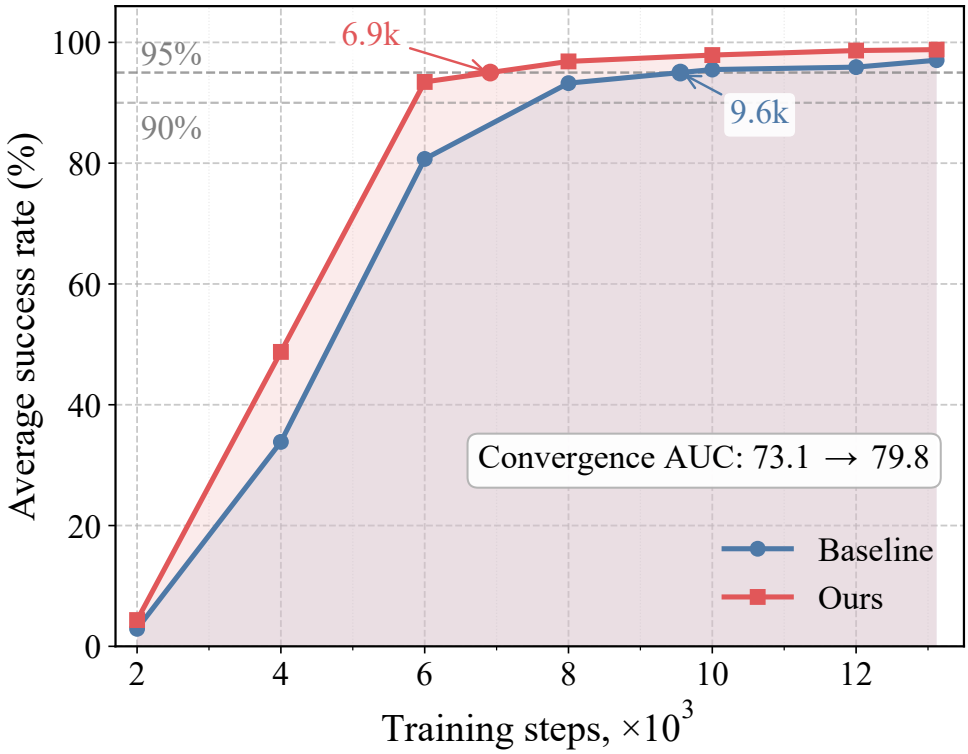


Figure 6. Training performance comparison on LIBERO.

in these two settings. With the full RoboTwin training set, DiffWAM-L instead achieves 93.28%, exceeding DiffWAM-S's 92.60% by 0.68 percentage points; both outperform FastWAM's 91.88%.

DiffWAM-S constrains optimization by freezing the independent difference-prediction module, reducing the number of parameters fitted to limited demonstrations while retaining difference supervision for the shared representation. This configuration balances adaptation with regularization, addressing the risk of overfitting to noise in small datasets. DiffWAM-L, in contrast, jointly optimizes all modules, providing greater flexibility to learn from the more diverse trajectories in the larger RoboTwin-Full training set. The performance comparison reflects this trade-off: DiffWAM-S achieves higher success rates in the two small-data settings, whereas DiffWAM-L performs better in the large-data setting. These results support choosing DiffWAM-S when training data are limited and DiffWAM-L when more data are available. Task-level results for full-data RoboTwin 2.0 training are provided in the appendix.

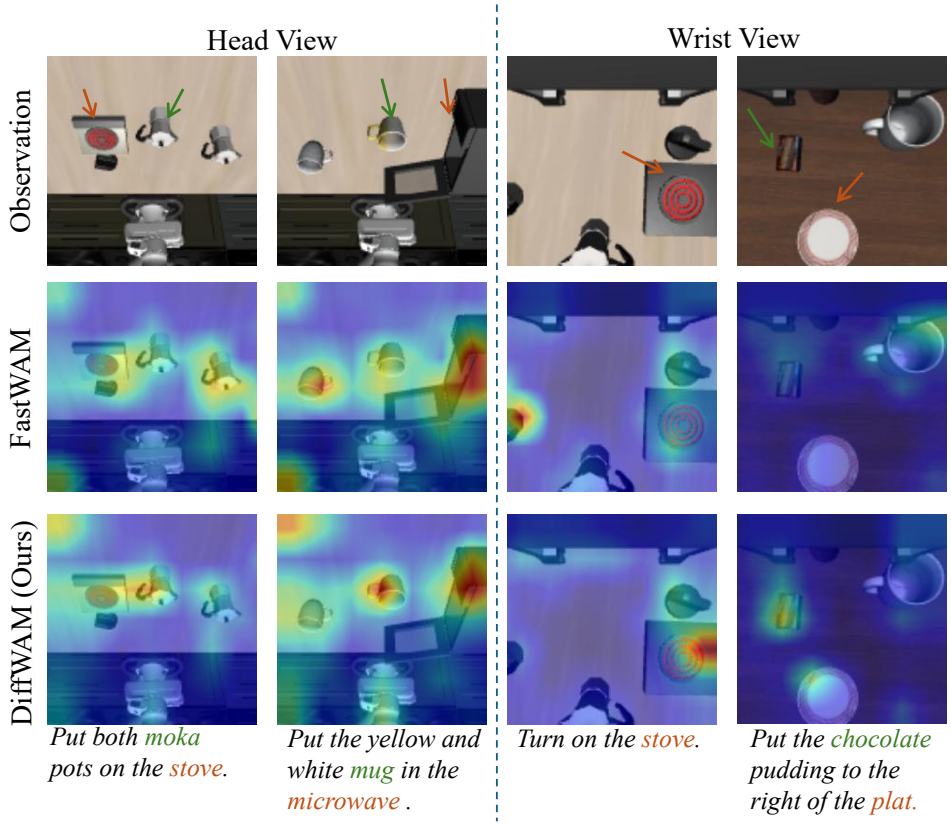


Figure 7. Attention visualizations. Arrows indicate target regions specified by the language instruction.

Setting	FastWAM	DiffWAM-L	DiffWAM-S
LIBERO	97.05	97.70	**98.80**
RoboTwin-Clean	88.47	87.72	**93.13**
RoboTwin-Full	91.88	**93.28**	92.60

Table 7. Success rates (%) of DiffWAM-L and DiffWAM-S. DiffWAM-L updates the LDB together with the remaining modules during its joint training stage; DiffWAM-S freezes the independent LDB parameters while retaining the diff loss. RoboTwin-Clean and RoboTwin-Full use clean-scene and full training data, respectively; both are evaluated in the clean setting.

Training Efficiency

Figure 6 compares the training dynamics of DiffWAM and FastWAM (baseline) on LIBERO. As shown in Figure 6, DiffWAM improves substantially faster during the early stage of training and reaches a 95% average success rate after only 6.9k steps, whereas FastWAM requires 9.6k steps. This corresponds to a 28.1% reduction in the number of optimization steps needed to reach the same performance level. DiffWAM also increases the convergence AUC from 73.1 to 79.8 and achieves a higher final success rate. These results indicate that visual difference supervision provides a more informative learning signal, enabling the policy to acquire task-relevant representations with fewer optimization steps while also improving its asymptotic performance.

4.4 Ablation Studies

Architecture Ablation

Table 5 examines whether the gains of DiffWAM arise from difference supervision alone or also benefit from a dedicated Diff Expert. Starting from the World Expert baseline (FastWAM), adding the diff loss raises average success from 97.05% to 97.75%. The clearest gain is on LIBERO-Long, where success increases from 92.40% to 95.80% (+3.40 points), suggesting that explicit difference supervision is particularly useful for tasks requiring sustained execution over multiple steps. However, loss-only supervision does not improve every suite: Spatial and Goal decrease slightly, while Object reaches 100.00%.

Retaining the diff loss and introducing the separate Diff Expert further raises average success to 98.80% (+1.05 points over loss-only supervision). This additional gain is concentrated on Goal and Long, which improve by 1.60 and 2.60 points, respectively, while Spatial recovers by 0.20 points and Object changes only slightly. Relative to FastWAM, the full architecture improves Long by 6.00 points and Goal by 1.20 points, indicating that its advantage is most pronounced in long-horizon execution and goal-conditioned manipulation. These results suggest that a dedicated difference-prediction pathway helps realize the benefits of difference supervision beyond simply adding an auxiliary loss. Thus, the diff loss provides a measurable improvement, and the Diff Expert contributes a further gain; both are necessary to attain the full performance of DiffWAM in this ablation.

Number of LDB Layers

We first investigate the effect of the LDB depth M on LIBERO. All DiffWAM entries in Table 4 use the DiffWAM-S configuration, with the independent LDB parameters frozen and the diff loss retained. All tested LDB depths outperform the FastWAM baseline on average. Increasing M from 5 to 10 improves the average success rate from 98.20% to 98.80%, with particularly strong gains on LIBERO-Goal and LIBERO-Long. Further increasing the depth to 15 or 20 layers does not yield additional improvements. These results favor a moderate depth for the difference-prediction pathway; we therefore use $M = 10$ as the default configuration.

Visual Expert Configurations

We next compare the visual expert configurations used during training. Table 6 considers Only World Expert, Only Diff Expert, and World + Diff Expert. These names refer to the visual prediction pathways; the Observation Encoder and Action Expert are retained for action learning in all three configurations. Only World Expert learns future-state prediction, Only Diff Expert learns latent-difference prediction, and World + Diff Expert jointly learns both.

As shown in Table 6, Only World Expert uses the measured FastWAM baseline and achieves an average success rate of 97.05%, while Only Diff Expert obtains 96.20%. World + Diff Expert improves the average success rate to 98.80%, outperforming the two single-expert configurations by 1.75 and 2.60 percentage points, respectively. The improvement is especially pronounced on LIBERO-Long, where the combined configuration reaches 98.40%, compared with 92.40% for Only World Expert and 89.80% for Only Diff Expert.

These results support the complementary roles of the two visual experts: the World Expert models future scene states, while the Diff Expert explicitly captures inter-frame visual differences. Jointly learning these capabilities enhances the representation used for action generation, particularly on long-horizon tasks.

Controlling for Additional Training

To distinguish the effect of difference supervision from that of additional optimization, we initialize FastWAM and DiffWAM from the same task-trained FastWAM checkpoint and train both for one additional epoch on clean demonstrations under matched training conditions. As shown in Table 8, DiffWAM achieves 93.13% clean success, compared with 88.47% for continued FastWAM training, a gain of 4.66 percentage points. This comparison supports an improvement beyond additional training alone and complements the visual-expert ablation in Table 6. The shared-checkpoint initialization isolates the effect of the modified training objective under a matched additional training budget.

Method	Additional training	Clean success (%)
FastWAM (continued training)	1 epoch	88.47
DiffWAM (Ours)	1 epoch	**93.13**

Table 8. **Controlling for additional training on RoboTwin 2.0.** Both methods start from the same task-trained FastWAM checkpoint and receive one additional epoch of training on clean demonstrations under matched conditions. This experiment is separate from the clean-to-random study in Table 3.

4.5 Visualization

Figure 7 visualizes the attention responses of DiffWAM from the head and wrist camera views. The attention maps retain fine-grained scene texture, while assigning stronger responses to the objects being manipulated. This indicates that DiffWAM can preserve detailed visual context without dispersing its representation uniformly over the scene; instead, it selectively concentrates attention on action-relevant objects. Overall, the visualization supports our design goal of learning representations that balance rich scene understanding with focused perception of the manipulated object.

5 Conclusion

In this work, we proposed DiffWAM, a World Action Model that enhances world representations by jointly learning future-state prediction, visual difference prediction, and action generation. Its Mixture-of-Experts architecture combines a shared Observation Encoder with dedicated World, Diff, and Action Experts. A structured unidirectional attention mask allows each expert to access the current-observation representation while separating their prediction pathways. At inference, the World and Diff Experts are removed, retaining only the Observation Encoder and Action Expert without additional inference overhead.

Experiments on LIBERO, RoboTwin 2.0, and real-world manipulation demonstrate the effectiveness of joint state and visual difference modeling. DiffWAM-S achieves average success rates of 98.8% on LIBERO and 88.3% on the real-world tasks, while DiffWAM-L achieves 92.65% on RoboTwin 2.0. Comparisons across data settings favor DiffWAM-S for smaller training sets and DiffWAM-L for the full RoboTwin training set, supporting the choice of training configuration according to data availability. An additional clean-to-random evaluation on RoboTwin 2.0 improves success from 1.50% to 5.27%, and a separate matched checkpoint-continuation control improves clean success from 88.47% to 93.13%. Ablation studies further confirm the complementary roles of the World and Diff Experts. Future work will explore large-scale pretraining and extend state and visual difference modeling to more diverse manipulation settings.

References

[1] Haoran Li, Yuhui Chen, Wenbo Cui, Weiheng Liu, Kai Liu, et al. Survey of vision-language-action models for embodied manipulation, 2025. URL https://arxiv.org/abs/2508.15201.

[2] Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, et al. A survey on vision-language-action models: An action tokenization perspective, 2025. URL https://arxiv.org/abs/2507.01925.

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, et al. RT-1: Robotics transformer for real-world control at scale, 2023.

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control, 2023.

[5] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, et al. OpenVLA: An open-source vision-language-action model, 2024.

[6] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025.

[7] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, et al. π_0: A vision-language-action flow model for general robot control, 2026.

[8] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization, 2025.

[9] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, et al. GR00T N1: An open foundation model for generalist humanoid robots, 2025.

[10] Siyin Wang, Junhao Shi, Zhaoyang Fu, Xinzhe He, Feihong Liu, et al. World action models: The next frontier in embodied ai, 2026. URL https://arxiv.org/abs/2605.12090.

[11] Qiuhong Shen, Shihua Zhang, Yue Liao, Qi Li, Zhenxiong Tan, et al. World action models: A survey, 2026. URL https://arxiv.org/abs/2606.20781.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024.

[13] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, et al. Motus: A unified latent action world model, 2025.

[14] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, et al. Causal world modeling for robot control, 2026.

[15] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination?, 2026.

[16] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, et al. FlowWAM: Optical flow as a unified action representation for world action models, 2026. URL https://arxiv.org/abs/2607.13017.

[17] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, et al. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets, 2025. URL https://arxiv.org/abs/2504.02792.

[18] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, et al. World action models are zero-shot policies, 2026. URL https://arxiv.org/abs/2602.15922.

[19] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, et al. Oa-wam: Object-addressable world action model for robust robot manipulation, 2026. URL https://arxiv.org/abs/2605.06481.

[20] Jialei Chen, Kai Wang, Kang Chen, Shuaihang Chen, Feng Gao, et al. Lawam: Latent world action models for efficient dynamics-aware robot policies, 2026. URL https://arxiv.org/abs/2606.15768.

[21] Jian Zhu, Jianjun Zhang, Taiyi Su, Tianbin Liu, Zhangyuan Wang, et al. Dswam: A dual-system world action foundation model for fine-grained robot manipulation, 2026. URL https://arxiv.org/abs/2607.04927.

[22] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning, 2023.

[23] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025.

[24] Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision–language–action models for embodied ai. *IEEE Transactions on Neural Networks and Learning Systems*, 37(7):3031–3051, 2026.

[25] Kento Kawaharazuka, Jihoon Oh, Jun Yamada, Ingmar Posner, and Yuke Zhu. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 13:162467–162504, 2025.

[26] Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems*, 2024.

[27] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, et al. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11), 2025.

[28] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, et al. Rdt-1b: A diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, 2025.

[29] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution, 2026.

[30] Rui Shao, Wei Li, Lingsen Zhang, Renshan Zhang, Zhiyang Liu, et al. Large vlm-based vision-language-action models for robotic manipulation: A survey, 2025. URL https://arxiv.org/abs/2508.13073.

[31] Xinqing Li, Xin He, Le Zhang, Min Wu, Xiaoli Li, and Yun Liu. A comprehensive survey on world models for embodied ai, 2026. URL https://arxiv.org/abs/2510.16732.

[32] Bohan Hou, Gen Li, Jindou Jia, Tuo An, Xinying Guo, et al. World model for robot learning: A comprehensive survey, 2026. URL https://arxiv.org/abs/2605.00080.

[33] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, et al. Gigaworld-policy: An efficient action-centered world-action model, 2026.

[34] Fangqi Zhu, Hongtao Wu, Song Guo, Yuxiao Liu, Chilam Cheang, and Tao Kong. Irasim: A fine-grained world model for robot manipulation. In *IEEE/CVF International Conference on Computer Vision*, 2025.

[35] Yichao Shen, Fangyun Wei, Zhiying Du, Yaobo Liang, Yan Lu, et al. Videovla: Video generators can be generalizable robot manipulators, 2025.

[36] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, et al. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025.

[37] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, et al. Worldvla: Towards autoregressive action world model, 2025.

[38] John Won, Kyungmin Lee, Huiwon Jang, Dongyoung Kim, and Jinwoo Shin. Dual-stream diffusion for world-model augmented vision-language-action model, 2026. URL https://arxiv.org/abs/2510.27607.

[39] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, et al. FLARE: Robot learning with implicit world modeling, 2025. URL https://arxiv.org/abs/2505.15659.

[40] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, et al. Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge, 2025. URL https://arxiv.org/abs/2507.04447.

[41] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, et al. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models, 2025. URL https://arxiv.org/abs/2503.22020.

[42] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. Flowdreamer: A rgb-d world model with flow-based motion representations for robot manipulation, 2025. URL https://arxiv.org/abs/2505.10075.

[43] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, et al. DINOv3, 2025. URL https://arxiv.org/abs/2508.10104.

[44] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, et al. Unified 4d world action modeling from video priors with asynchronous denoising, 2026. URL https://arxiv.org/abs/2604.26694.

[45] Liaoyuan Fan, Zetian Xu, Chen Cao, Wenyao Zhang, Mingqi Yuan, et al. AIM: Intent-aware unified world action modeling with spatial value maps, 2026. URL https://arxiv.org/abs/2604.11135.

[46] Lu Qiu, Yizhuo Li, Yi Chen, Yuying Ge, Yixiao Ge, et al. Making foresight actionable: Repurposing representation alignment in world action models, 2026. URL https://arxiv.org/abs/2606.12217.

[47] Wenyao Zhang, Bozhou Zhang, Zekun Qi, Wenjun Zeng, Xin Jin, and Li Zhang. Disentangled robot learning via separate forward and inverse dynamics pretraining, 2026. URL https://arxiv.org/abs/2604.16391.

[48] Yajie Li, Bozhou Zhang, Chun Gu, Zipei Ma, Jiahui Zhang, Jiankang Deng, Xiatian Zhu, and Li Zhang. From imagined futures to executable actions: Mixture of latent actions for robot manipulation, 2026. URL https://arxiv.org/abs/2605.12167.

[49] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, et al. Wan: Open and advanced large-scale video generative models, 2025. URL https://arxiv.org/abs/2503.20314.

[50] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, et al. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023. URL https://arxiv.org/abs/1910.10683.

[51] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL https://arxiv.org/abs/1312.6114.

[52] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL https://arxiv.org/abs/2210.02747.

[53] David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, et al. Flow matching policy gradients, 2025. URL https://arxiv.org/abs/2507.21053.

[54] Daojie Peng, Fulong Ma, Jiahang Cao, Qiang Zhang, Xupeng Xie, et al. Attena+: Rectifying action inequality in robotic foundation models, 2026. URL https://arxiv.org/abs/2605.13548.

[55] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 79115–79135. PMLR, 2025. URL https://proceedings.mlr.press/v267/zhou25t.html.

[56] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, et al. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation, 2026. URL https://arxiv.org/abs/2508.19236.

[57] Yuyang Zhang, Wenyao Zhang, Zekun Qi, He Zhang, Haitao Lin, et al. Imagewam: Do world action models really need video generation, or just image editing?, 2026. URL https://arxiv.org/abs/2606.19531.

[58] Yi Yang, Zhihong Liu, Siqi Kou, Yiyang Chen, Yanzhe Hu, et al. World-language-action model for unified world modeling, language reasoning, and action synthesis, 2026. URL https://arxiv.org/abs/2606.05979.

[59] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025. URL https://arxiv.org/abs/2510.10274.

[60] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning, 2026. URL https://arxiv.org/abs/2602.11236.

[61] Qiuyue Wang, Mingsheng Li, Jian Guan, Jinhui Ye, Sicheng Xie, et al. Qwen-vla: Unifying vision-language-action modeling across tasks, environments, and robot embodiments, 2026. URL https://arxiv.org/abs/2605.30280.

[62] StarVLA Community. StarVLA: A lego-like codebase for vision-language-action model developing, 2026. URL https://arxiv.org/abs/2604.05014.

[63] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, Ru Ying, Ran Zheng, and Yao Mu. AHA-WAM: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing, 2026. URL https://arxiv.org/abs/2606.09811.

[64] Xinyu Guo, Chao Jing, Li Yi, Xingyu Chen, and Zhengxing Wu. PINT: A unified model with shared predictive intent for robot manipulation, 2026. URL https://arxiv.org/abs/2610.16355.

A Additional Experimental Details

The key configurations for the three experimental settings are summarized in Table 9. We use DiffWAM-S for the main LIBERO and real-world experiments and DiffWAM-L for the main RoboTwin 2.0 experiments.

Setting	LIBERO	RoboTwin 2.0	Real
Model configuration	DiffWAM-S	DiffWAM-L	DiffWAM-S
Training hardware	8× A100	8× A100	8× A100
Grad. accumulation	2	2	2
Weight decay	1×10^{-2}	1×10^{-2}	1×10^{-2}
LR schedule	Cosine	Cosine	Cosine
Learning rate	1×10^{-4}	1×10^{-5}	1×10^{-4}
Batch size	8	8	8
Training steps	1.3×10^4	3×10^4	6.3×10^4
Network depth N	30	30	30
Network depth M	10	10	10
Cameras	2	3	2
Video size	$[224, 448]$	$[384, 320]$	$[224, 576]$
Image resize	$[224, 224]$	$[240, 320]$	$[240, 320]$
Action dim.	7	14	7
Proprio dim.	8	14	7
Normalization	min/max	z-score	z-score
Validation split	0.0	0.01	0.0

Table 9. Implementation configurations for each experimental setting.

LIBERO

LIBERO [22] consists of four suites: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long. Each suite contains 10 tasks and provides 500 expert demonstrations, resulting in 40 tasks and 2,000 demonstrations in total. We conduct 100 rollouts per task, amounting to 4,000 evaluation trials, and report the average success rate for each suite and across all four suites. For this benchmark, DiffWAM-S is initialized from Wan2.2-5B and trained for 1.3×10^4 steps with a learning rate of 1×10^{-4} and a batch size of 8. The model uses two camera views, a 7-dimensional action space, an 8-dimensional proprioceptive input, min/max normalization, and network depths of $N = 30$ and $M = 10$.

RoboTwin 2.0

RoboTwin 2.0 [23] evaluates bimanual manipulation under both clean and heavily randomized environments. We adopt its multi-task setting with 50 tasks and use approximately 27k demonstration trajectories in total, comprising 2,500 demonstrations from clean scenes and 25,000 demonstrations with strong scene randomization. Each task is evaluated over 100 trials under both clean and randomized conditions, and we report the success rates for the two settings and their average. Due to computational resource constraints, DiffWAM-L is initialized from FastWAM for this benchmark and trained for 3×10^4 steps with a learning rate of 1×10^{-5} and a batch size of 8. The model uses three camera views, a 14-dimensional action space, a 14-dimensional proprioceptive input, z-score normalization, and network depths of $N = 30$ and $M = 10$.

Clean-to-random evaluation

The additional experiment in Table 3 uses clean demonstrations only during task training and evaluates the resulting policies under randomized RoboTwin 2.0 conditions without adaptation. These runs start task training without loading a task-trained

FastWAM checkpoint; training from scratch here refers to robot-task training and does not imply that all model parameters are randomly initialized. This experiment is distinct from the full-data training configuration above, which includes randomized demonstrations. Clean/Random success is 70.2%/1.50% for our FastWAM baseline and 69.7%/5.27% for DiffWAM. We evaluate DiffWAM over 30 trials per task in each condition, totaling 1,500 trials per condition. It succeeds in 1,045 clean trials and 79 randomized trials, yielding unrounded success rates of 1,045/1,500 and 79/1,500. The Clean/Random average is computed before rounding as 1,124/3,000, or 37.47% when rounded to two decimal places. The corresponding FastWAM average is 35.85%. Clean results from the separate checkpoint-continuation experiment are not paired with these values.

Checkpoint-continuation control

For Table 8, FastWAM and DiffWAM instead share the same task-trained FastWAM checkpoint as their starting point, with the additional Diff pathway introduced for DiffWAM. Both receive one additional epoch of training on clean demonstrations under matched conditions and are evaluated in the clean setting. The reported success rates are 88.47% and 93.13%, respectively. This comparison controls for the additional training budget and is separate from both the clean-to-random evaluation and the full-data RoboTwin result. No mechanistic claim about overfitting is made from the FastWAM score alone.

Real-world

We evaluate DiffWAM-S on the ALOHA-AgileX platform,[1] using approximately 1.1k demonstration trajectories in total across Place Pens, Store Block, and Stack Cups and reporting the execution success rate over 20 trials per task for each method (60 trials per method in total). For these experiments, DiffWAM-S is initialized from Wan2.2-5B and trained for 6.3×10^4 steps with a learning rate of 1×10^{-4} and a batch size of 8. The model uses two camera views, a 7-dimensional action space, a 7-dimensional proprioceptive input, z-score normalization, and network depths of $N = 30$ and $M = 10$.

B Additional Experimental Results

This section describes the comparison methods and provides task-level RoboTwin 2.0 results, real-world analyses, visualizations of state and visual difference prediction, and supplementary training-performance results.

B.1 Comparison Methods

We compare DiffWAM with representative vision language action (VLA) and world action model (WAM) approaches. VLA methods directly map visual-language observations to actions, while WAM methods additionally model action-conditioned dynamics.

VLAs

OpenVLA [5] and OpenVLA-O [6] are an open VLA baseline and its optimized variant; MemoryVLA [56] augments VLA control with memory. $\pi 0$ [7] and $\pi 0.5$ [8] are flow-based VLA models, with the latter targeting open-world generalization. GR00T-N1 [9] is a generalist humanoid foundation model, X-VLA [59] targets cross-embodiment transfer, ABot-M0 [60] learns an action manifold, and QwenVLA [61] unifies VLA modeling across tasks and embodiments.

WAMs

Motus [13] models latent actions in a unified world model; LingBot-VA [14] uses causal world modeling; and FastWAM [15] is the WAM baseline used for RoboTwin 2.0 initialization. ImageWAM [57], X-WAM [44], and WLA [58] respectively explore image editing based WAM, 4D world action modeling, and joint world language action modeling.

[1] https://github.com/agilexrobotics/aloha-agilex

B.2 RoboTwin 2.0 Results

Full data training

Table 12 reports task-level results when training with the full RoboTwin 2.0 data. DiffWAM-L achieves 93.28% success in clean scenes and 92.02% under randomization, for an average of 92.65%. This improves over the reproduced FastWAM baseline by 1.55 percentage points overall, with gains in both clean and randomized conditions (91.88% to 93.28% and 90.32% to 92.02%, respectively). At the task level, DiffWAM-L is competitive across a broad range of manipulation skills and reaches or ties the best result on many tasks, indicating that explicitly modeling inter-frame visual differences improves performance without sacrificing robustness to scene variation.

B.3 Independent Training Runs and Uncertainty

For each of LIBERO and the full-data and clean-only RoboTwin 2.0 settings, we independently train our method three times with different random seeds and evaluate each resulting model using the corresponding benchmark protocol. Table 10 reports the score from each training run, together with the mean and sample standard deviation across the three runs. The resulting scores are $92.52 \pm 0.11\%$ on full-data RoboTwin 2.0, $98.67 \pm 0.12\%$ on LIBERO, and $37.39 \pm 0.08\%$ for the Clean/Random average in the clean-only RoboTwin setting.

Benchmark	Metric	Run 1	Run 2	Run 3	Mean $\pm$ Std.
RoboTwin 2.0	Avg. (full-data)	92.49	92.65	92.43	92.52 ± 0.11
LIBERO	Avg.	98.60	98.80	98.60	98.67 ± 0.12
RoboTwin 2.0	Avg. (clean-only)	37.32	37.38	37.47	37.39 ± 0.08

Table 10. **Independent training runs of DiffWAM.** Success rates (%) from three models trained independently with different random seeds and evaluated separately. Run 1–3 denote independent training-and-evaluation runs. We report their arithmetic mean and sample standard deviation ($n = 3$, denominator $n - 1$). RoboTwin Avg. is the mean of Clean and Random success; full-data and clean-only denote the respective training settings. The clean-only average is not the Random-only OOD success rate. LIBERO Avg. is the average over its four suites.

The reported metrics exhibit limited variability across the three independently trained models, supporting the consistency of the method across training seeds in these settings. Each observation corresponds to a separate training run followed by evaluation, so the standard deviations summarize variation across complete training-and-evaluation runs. Main-table scores correspond to individual runs and are reported separately from the three-run means. In particular, the clean-only RoboTwin metric averages Clean and Random success and should not be interpreted as a Random-only success rate.

B.4 Real-world Results

Our real-world study evaluates DiffWAM-S on three manipulation tasks: Place Pens, Store Block, and Stack Cups. We use approximately 1.1k demonstration trajectories in total and train for 63k optimization steps. Each method is evaluated over 20 trials per task, totaling 60 trials across the three tasks. As shown in Figure 5 of the main paper, DiffWAM-S achieves success rates of 90%, 85%, and 90%, respectively, compared with 80%, 70%, and 70% for FastWAM. These rates correspond to 18/20, 17/20, and 18/20 successful trials for DiffWAM-S, compared with 16/20, 14/20, and 14/20 for FastWAM. The average success rate increases from 73.3% (44/60) to 88.3% (53/60), an absolute gain of 15.0 percentage points. The improvements across all three tasks support the effectiveness of joint state and visual difference modeling in this real-world setting. Broader evaluation across objects, environments, and embodiments remains necessary to establish generality.

Metric	f_1	f_2	Δf_1	Δf_2
PSNR⇑	37.94	30.79	-	-
SSIM⇑	0.9851	0.9701	-	-
Latent Error⇓	-	-	0.0055	0.0136

Table 11. Future-state and visual difference prediction. PSNR and SSIM evaluate predicted future observations; latent error evaluates predicted differences between consecutive frame latents.

B.5 Visualization Results

Task Level Output Visualization

Figure 9 qualitatively visualizes the outputs of the World and Diff Experts on representative manipulation tasks. Both experts receive the same noised future-frame latents: the World Expert predicts future states through denoising, while the Diff Expert directly regresses differences between consecutive ground-truth frame latents. The predicted future observations preserve scene structure and reflect the progression of the manipulation trajectory. The visualized diff outputs highlight localized variation around the robot and manipulated objects, illustrating the complementary information captured by state and visual difference prediction. These differences describe inter-frame visual variation and do not exclusively isolate changes caused by robot actions.

Table 11 quantitatively evaluates the two prediction branches. For the first and second future observations, the World Expert achieves PSNR values of 37.94 and 30.79 and SSIM values of 0.9851 and 0.9701, respectively. The Diff Expert obtains latent prediction errors of 0.0055 and 0.0136 for Δf_1 and Δf_2. These results characterize the model's ability to predict both future scene states and their inter-frame latent differences.

B.6 Training Performance Analysis

Figure 8 reports the per-suite improvement of DiffWAM over the FastWAM baseline throughout training. DiffWAM achieves non-negative gains at all earlier evaluation checkpoints and improves the final average success rate. The advantage is strongest in the early and middle stages: the average gain reaches 14.9 points at 4k steps and 12.8 points at 6k steps. At 4k steps, the improvements on LIBERO-Object and LIBERO-Goal are 28.2 and 18.2 points, respectively; LIBERO-Long reaches its largest gain of 37.8 points at 6k steps. Although the gaps narrow as training proceeds, DiffWAM improves the final average success rate by 1.75 percentage points, with gains on Goal and Long, a tie on Object, and a 0.2-point decrease on Spatial, including a 6.0-point gain on LIBERO-Long. These results show that DiffWAM provides consistent training improvements, especially for the more challenging goal-conditioned and long-horizon tasks.

C Limitations and Future Work

C.1 Limitations

Several limitations remain. First, the independent M-layer Diff Expert pathway introduces additional computation and memory during training. Freezing the LDB in DiffWAM-S does not eliminate this cost, since diff predictions are still computed and their gradients propagate into the shared representation. Both the World and Diff Experts are removed at inference, so these visual prediction pathways add no online action-generation overhead. Second, the difference target captures inter-frame latent variation without explicitly distinguishing changes caused by robot actions from camera motion, illumination changes, or movement of unrelated objects. This ambiguity may be more significant in dynamic, unstructured environments. Third, the clean-to-random result establishes a benefit under the evaluated RoboTwin perturbations, while absolute Random success remains low and generalization to other shifts is not established. The experiments cover a limited range of data scales and



Figure 8. Supplementary per-suite training improvements of DiffWAM over the FastWAM baseline on LIBERO at each evaluation checkpoint.

embodiments, and DiffWAM is not pretrained on a large-scale robot dataset. Broader evaluation is needed to establish how the benefits of DiffWAM-S and DiffWAM-L generalize across training-data regimes and deployment settings.

C.2 Future Work

Future work will extend DiffWAM in three corresponding directions. First, more efficient Diff Expert designs, such as lightweight prediction heads or selective temporal token processing, could reduce the additional training cost and enable scaling to larger backbones, longer horizons, and richer multi-view observations. Second, we will investigate action-conditioned, motion-aware, and object-centric constraints to disentangle action-controllable changes from nuisance dynamics; multi-scale and multi-horizon diff prediction may further capture both fine-grained contact dynamics and longer-term task progress. Third, large-scale pretraining on diverse robot interaction data is a promising direction for learning more transferable visual and action priors before downstream adaptation. Evaluating this setting across different embodiments, camera configurations, objects, and long-horizon real-world tasks will be important for establishing the generality of diff prediction.

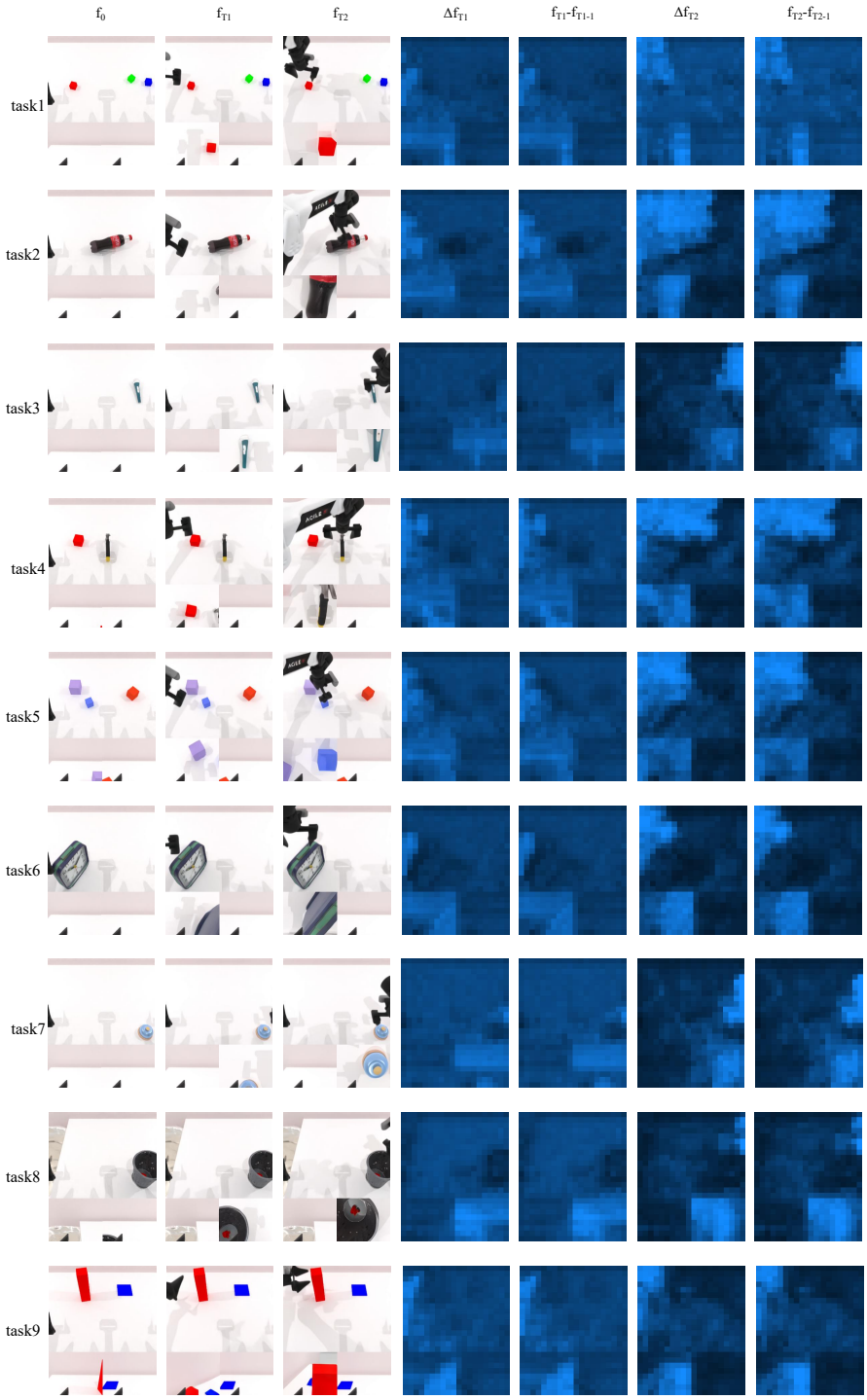


Figure 9. Qualitative visualization of future-state and visual difference prediction by the World and Diff Experts on representative manipulation tasks.

Task	π0.5		Motus		LingBot-VA		GigaWorld-Policy		X-WAM		FastWAM		DiffWAM-L	
	Clean	Random	Clean	Random	Clean	Random	Clean	Random	Clean	Random	Clean	Random	Clean	Random
Adjust Bottle	**100**	99	89	93	90	94	**100**	**100**	**100**	99	**100**	**100**	**100**	**100**
Beat Block Hammer	96	93	95	88	96	98	86	86	98	96	**99**	90	97	**100**
Blocks Ranking RGB	92	85	99	97	99	98	92	96	99	95	**100**	**100**	**100**	**100**
Blocks Ranking Size	49	26	75	63	94	96	44	48	76	82	94	**100**	**100**	97
Click Alarmclock	98	89	**100**	**100**	99	**100**	**100**	**100**	98	99	**100**	**100**	**100**	**100**
Click Bell	99	66	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
Dump Bin Bigbin	92	97	95	91	89	96	92	**100**	90	96	97	97	**100**	**100**
Grab Roller	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
Handover Block	66	57	86	73	**99**	78	80	80	88	79	95	83	97	**90**
Handover Mic	98	97	78	63	94	96	72	72	88	89	99	**100**	**100**	**100**
Lift Pot	18	17	38	38	40	28	16	12	**100**	99	58	97	**100**	**100**
Move Can Pot	96	85	96	**99**	**100**	**99**	98	98	80	84	**100**	80	90	87
Move Playingcard Away	51	55	34	74	94	97	76	78	**100**	98	90	**100**	**100**	**100**
Move Stapler Pad	84	61	93	96	99	**99**	90	90	67	70	**100**	53	73	57
Hanging Mug	96	84	**100**	96	**100**	**99**	78	72	46	55	**100**	53	63	57
Open Laptop	56	42	83	85	91	79	92	82	96	97	77	**100**	**100**	**100**
Open Microwave	90	96	95	91	92	94	96	**98**	89	92	**98**	37	73	53
Pick Diverse Bottles	34	77	**95**	91	82	86	74	66	91	**92**	62	87	93	87
Pick Dual Bottles	81	71	90	91	89	82	82	70	99	**100**	80	97	**100**	97
Place A2B Left	93	63	96	90	**100**	**99**	86	86	90	87	**100**	93	93	97
Place A2B Right	87	82	88	79	**97**	93	94	88	92	89	95	97	90	**100**
Place Bread Basket	87	84	91	87	**97**	**95**	90	92	90	91	93	87	87	90
Place Bread Skillet	77	64	91	94	**97**	95	82	82	90	96	91	**97**	**97**	93
Place Can Basket	85	66	86	83	**95**	**90**	94	**90**	84	82	90	60	63	77
Place Cans Plasticbox	94	87	98	98	97	95	98	96	99	98	71	**100**	**100**	**100**
Place Container Plate	62	62	81	76	81	84	78	74	98	**100**	99	97	**100**	**100**
Place Dual Shoes	94	84	98	94	**100**	99	**100**	**100**	83	81	96	87	93	87
Place Empty Cup	99	95	98	99	99	97	98	96	98	99	94	**100**	**100**	**100**
Place Fan	75	75	93	87	94	89	96	84	84	92	**100**	**97**	97	93
Place Burger Fries	**100**	99	99	98	**100**	**100**	90	90	97	99	96	**100**	**100**	97
Place Mouse Pad	87	85	91	87	**99**	93	92	**94**	84	86	96	87	87	83
Place Object Basket	60	39	66	68	93	**96**	88	90	85	87	83	77	**97**	87
Place Object Scale	80	76	81	87	91	88	90	92	**93**	89	89	93	**93**	**97**
Place Object Stand	86	80	88	85	96	95	88	80	**97**	**96**	90	90	93	87
Place Phone Stand	91	85	98	97	99	96	**100**	**98**	75	80	90	97	97	97
Move Pillbottle Pad	81	81	87	86	**97**	97	82	72	96	**98**	**97**	90	**97**	97
Place Shoe	92	93	**99**	97	98	98	98	96	97	**99**	96	97	97	97
Press Stapler	87	83	93	98	85	82	96	96	94	90	90	**100**	**97**	**100**
Put Bottles Dustbin	84	79	81	79	87	91	72	70	85	**95**	**95**	83	87	83
Put Object Cabinet	80	79	88	71	85	87	74	74	66	76	**94**	**93**	93	90
Rotate Qrcode	89	87	89	73	**96**	**91**	90	84	84	83	93	90	87	90
Scan Object	72	65	67	66	96	91	60	64	86	79	89	**93**	**97**	87
Shake Bottle	99	97	**100**	97	**100**	97	**100**	98	99	99	**100**	**100**	**100**	**100**
Shake Bottle Horizontally	99	99	**100**	98	**100**	99	**100**	**100**	**100**	99	**100**	**100**	**100**	**100**
Stack Blocks Three	91	76	91	95	**99**	98	70	78	97	95	95	97	93	**100**
Stack Blocks Two	97	**100**	**100**	98	**100**	98	**100**	94	**100**	**100**	**100**	**100**	**100**	**100**
Stack Bowls Three	77	71	79	**87**	86	83	70	72	**88**	82	80	**87**	77	**87**
Stack Bowls Two	95	96	**98**	98	94	98	96	92	**98**	98	92	**100**	93	97
Stamp Seal	79	55	93	92	**96**	97	**96**	**98**	93	95	90	93	93	93
Turn Switch	62	54	**84**	78	44	45	82	**84**	61	72	61	60	80	70
Overall	82.74	76.76	88.66	87.02	92.90	91.50	86.36	85.04	89.76	90.68	91.88	90.32	**93.28**	**92.02**

Table 12. Task-level RoboTwin 2.0 results. Columns report clean/random success rates for each method.